

**A PROJECT REPORT ON**

**Optimizing Manpower Allocation in Rail Construction**

SUBMITTED TO THE SAVITRIBAI PHULE UNIVERSITY, PUNE IN  
THE PARTIAL FULFILLMENT OF THE REQUIREMENTS FOR THE  
AWARD OF THE DEGREE

OF

**BACHELOR OF ENGINEERING**  
**(COMPUTER ENGINEERING)**

**SUBMITTED BY**

Saniya Atalatti  
Shatakshi Chaudhari  
Arnav Desai  
Hatim Talwara

Exam No: 41306  
Exam No: 41313  
Exam No: 41317  
Exam No: 41329



**DEPARTMENT OF COMPUTER ENGINEERING**  
**Pune Institute of Computer Technology**  
**Dhankawadi, Pune - 411043**

**SAVITRIBAI PHULE PUNE UNIVERSITY**  
**2024-2025**



## CERTIFICATE

This is to certify that the project report entitled  
**Optimizing Manpower Allocation in Rail Construction**

Submitted by

Saniya Atalatti  
Shatakshi Chaudhari  
Arnav Desai  
Hatim Talwara

Exam No: 41306  
Exam No: 41314  
Exam No: 41317  
Exam No: 41329

are a bonafide student of this institute and the work has been carried out by them under the supervision of **Dr. Girish Potdar** and it is approved for the partial fulfillment of the requirement of Savitribai Phule Pune University, for the award of the degree of **Bachelor of Engineering** (Computer Engineering).

**Dr. Girish Potdar**  
Internal Guide  
Dept. of Computer Engg.

**Dr. G. V. Kale**  
HOD  
Dept. of Computer Engg.

Place: Pune Date: 25/04/2025

## ACKNOWLEDGEMENT

*It gives us great pleasure to present the project report on ‘Optimizing Manpower Allocation in Rail Construction’.*

*We would like to thank our project guide **Dr. Girish Potdar** for giving us all the help and guidance we needed. We are really grateful to them for their kind support and valuable suggestions.*

*We are also grateful to **Dr. G. V. Kale**, Head of Computer Engineering Department, PICT for her indispensable support and suggestions throughout the project. We also express our gratitude to **Dr. Girish Potdar** BE project coordinator, for her timely guidance and support.*

*Our special thanks to our external guide **Sameer Hase**, Project Manager, DB Engineering and Consulting, for their continued support and valuable inputs during ideation and implementation phase of the project. We also thank our laboratory assistants for their valuable help in setting up the workstations. We are deeply thankful for the platform and resources that PICT offers us, allowing us to undertake this valuable project.*

Saniya Atalatti

Shatakshi Chaudhari

Arnav Desai

Hatim Talwara

(B.E. Computer Engg.)

## **ABSTRACT**

*In rapidly urbanizing regions, particularly in developing countries, the demand for efficient and sustainable transport systems has become critical. One of the most significant challenges facing urban transport infrastructure development, specifically in the rail construction industry, is the optimal management of human resources. This paper addresses the complexities involved in manpower allocation and resource management within the rail construction sector, focusing on the application of Operations Research techniques, particularly Linear Programming (LP), to improve efficiency and decision-making. The study analyzes various categories of labor, including skilled, medium-skilled, and unskilled workers, taking into account their availability, cost, and the specific workforce demands for key projects such as new railway construction, track maintenance, and station renovations. By integrating these variables into an LP model, the research provides a structured approach to minimize operational costs while ensuring adequate workforce distribution across multiple, simultaneous projects. The results demonstrate how effective manpower optimization can significantly enhance project timelines, reduce cost overruns, and contribute to the overall sustainability of the urban transport infrastructure. Additionally, the paper offers insights into overcoming the practical challenges of resource scarcity and cost fluctuations that are prevalent in the rail construction sector, with recommendations aimed at supporting policymakers and industry leaders in making informed, data-driven decisions. Through the application of this LP-based framework, the research underscores the potential for Operations Research to play a pivotal role in addressing the manpower challenges of large-scale infrastructure projects, ultimately contributing to the long-term efficiency and resilience of urban transportation systems.*

# Contents

|                                                           |           |
|-----------------------------------------------------------|-----------|
| <b>Introduction</b>                                       | <b>10</b> |
| 1.1 Overview                                              | 10        |
| 1.2 Motivation                                            | 10        |
| 1.3 Problem Definition and Objectives                     | 11        |
| 1.4 Project Scope & Limitations                           | 12        |
| 1.5 Methodology of Problem solving                        | 13        |
| <b>2 Literature Survey</b>                                | <b>15</b> |
| 2.1 Literature survey                                     | 15        |
| <b>3 Software Requirements Specification</b>              | <b>16</b> |
| 3.1 Software Requirements Specification                   | 17        |
| Assumptions and Dependencies                              | 18        |
| 3.2 Functional Requirements                               | 19        |
| Upload Railway Network Data (Stations, Routes)            | 20        |
| Define Optimization Objective (Shortest Path, Time, Cost) | 21        |
| Run Optimization Algorithm                                | 22        |
| Graphical Visualization of Network                        | 22        |
| Display Optimized Results (Text, Graph, Map)              | 22        |
| Download Optimized Output (CSV/Excel)                     | 23        |
| Reset & Rerun Optimization                                | 23        |
| 3.3 External Interface Requirements                       | 24        |
| Streamlit Web Interface                                   | 24        |
| File Upload Interface (CSV/Excel Input)                   | 25        |
| Interactive Widgets (Sliders, Dropdowns)                  | 25        |
| Graph & Map Visualization Interface                       | 26        |
| 3.4 Nonfunctional Requirements                            | 27        |
| Performance Requirements                                  | 28        |
| Safety Requirements & Security Requirements               | 28        |
| Usability and Accessibility                               | 28        |
| Availability and Deployment                               | 29        |
| 3.5 System Requirements                                   | 30        |
| Database Requirements                                     | 31        |
| Software Requirements                                     | 31        |
| Hardware Requirements                                     | 32        |
| <b>4 System Design</b>                                    | <b>37</b> |

|          |                                        |           |
|----------|----------------------------------------|-----------|
| 4.1      | System Architecture                    | 37        |
| 4.2      | UML Diagrams                           | 38        |
| 4.3      | Implementation Diagram                 | 39        |
| 4.4      | Deployment Diagram                     | 40        |
| <b>5</b> | <b>Project Plan</b>                    | <b>41</b> |
| 5.1      | Project Estimate                       | 42        |
| 5.1.1    | Reconciled Estimates                   | 43        |
| 5.1.2    | Project Resources                      | 44        |
| 5.2      | Risk Management                        | 45        |
| 5.2.1    | Risk Identification                    | 46        |
| 5.2.2    | Risk Analysis                          | 47        |
| 5.3      | Project Schedule                       | 48        |
| 5.3.1    | Project Task Set                       | 48        |
| 5.3.2    | Timeline Chart                         | 49        |
| 5.4      | Team Organization                      | 50        |
| 5.4.1    | Team structure                         | 51        |
| 5.4.2    | Management reporting and communication | 52        |
| <b>6</b> | <b>Other Specifications</b>            | <b>53</b> |
| 6.1      | Advantages                             | 53        |
| 6.2      | Limitations                            | 54        |
| 6.3      | Applications                           | 55        |
| <b>7</b> | <b>Conclusion &amp; Future Work</b>    | <b>56</b> |
| 7.1      | Conclusion                             | 57        |
| 7.2      | Future Work                            | 58        |
| <b>8</b> | <b>Appendix</b>                        | <b>59</b> |
| 8.1      | Appendix A                             | 60        |
| 8.1.1    | Technical Feasibility                  | 61        |
| 8.1.2    | Operational Feasibility                | 62        |
| 8.1.3    | Economic Feasibility                   | 62        |
| 8.2      | Appendix B                             |           |
| 8.2.1    | Survey Paper Details                   | 63        |
| 8.3      | Appendix C                             | 64        |
| 8.3.1    | Plagiarism Report                      | 64        |
| <b>9</b> | <b>References</b>                      | <b>65</b> |



# **CHAPTER 1**

## **INTRODUCTION**



## **1.1 Overview**

The rail construction industry in developing countries faces significant challenges in efficiently allocating manpower, managing resources, and controlling costs across various large-scale infrastructure projects. This study addresses these issues by applying Linear Programming (LP) to optimize workforce distribution, ensuring that each project, whether it be railway expansion, track maintenance, or station upgrades, receives the necessary labor while minimizing operational expenses. The LP-based model considers labor availability, skill levels, cost constraints, and time sensitivity, offering a comprehensive framework for decision-makers to enhance resource allocation. By focusing on practical solutions for managing workforce shortages, fluctuating labor costs, and project delays, this research provides actionable strategies to improve the efficiency and sustainability of rail infrastructure development.

## **1.2 Motivation**

The motivation behind this study stems from the growing need for efficient infrastructure development in developing countries, where large-scale rail construction projects are vital for economic growth and connectivity. However, these projects often face challenges such as labor shortages, cost overruns, and inefficient resource allocation, which can delay progress and increase financial strain. Traditional methods of workforce distribution and project management are often inadequate in addressing these complexities, leading to suboptimal outcomes. By introducing Linear Programming as a method to optimize manpower allocation, this study seeks to provide a more systematic, data-driven approach to resource management. The goal is to enhance project efficiency, reduce costs, and ensure timely completion, ultimately contributing to the advancement of critical infrastructure that supports broader societal and economic goals.

## **1.3 Problem Definition and Objectives**

In large-scale infrastructure projects like rail construction, efficient resource management is crucial to meeting deadlines and controlling costs. One of the key challenges faced in such projects is the optimal allocation of manpower across various tasks. Improper distribution of labor can lead to project delays, increased costs, and underutilization of available resources. Traditional methods of workforce management often rely on manual planning and estimation, which

are prone to errors and inefficiencies. This study seeks to address these issues by applying Linear Programming techniques to optimize manpower allocation, ensuring the efficient use of labor resources in rail construction projects. Company specifies its shortlisting criteria to get a list of eligible students, this criteria is specified by TNP Cell in a textual format in google form description. In Spite of this, ineligible students fill the form resulting in absurd data problems and TNP then again needs to refilter the data and send it to the company. During a company's placement drive, students face a lot of issues related to test platforms, login issues, etc. Students then con- tact TNP heads regarding the issues, it becomes difficult to handle a heap of queries from students on admin side.

The primary objective of this study is to develop and implement a Linear Programming model that optimizes manpower allocation in rail construction projects. The goal is to minimize project costs and time by ensuring the most efficient distribution of workers across different tasks, while considering project constraints such as budget limits, available workforce, and task deadlines. This research aims to demonstrate how mathematical optimization can be applied to enhance resource management in infrastructure projects, ultimately leading to more efficient project execution and reduced operational costs.

### 1.4 Project Scope & Limitations

This project focuses on optimizing manpower allocation in rail construction projects through the application of Linear Programming (LP). It will develop a model to ensure that labor resources are assigned to tasks in a way that minimizes project costs and meets deadlines. The model will take into account key factors such as workforce availability, task priorities, and budget constraints. The scope also includes a detailed analysis of real-world rail construction data to validate the effectiveness of the model. Additionally, this project aims to provide insights into how LP techniques can be applied in other large-scale infrastructure projects to improve overall resource efficiency.

#### **Limitations:**

1. **Data Accuracy:** The effectiveness of the Linear Programming model is highly dependent on the accuracy and availability of data related to workforce skills, project timelines, and cost estimates. Any inaccuracies in this data may affect the results.
2. **Assumption of Constant Parameters:** The model assumes that factors such as workforce availability, task duration, and costs remain constant throughout

the project. In real-world scenarios, these factors may fluctuate due to unforeseen events, causing deviations from the optimal solution.

3. **Limited Flexibility:** While Linear Programming is effective for solving deterministic problems, it may lack the flexibility to account for sudden changes in project scope or workforce dynamics, such as unexpected absenteeism or equipment failures.
4. **Computational Complexity:** For large-scale rail construction projects with numerous tasks and constraints, the computational complexity of the LP model could increase, leading to longer processing times and difficulty in real-time decision-making.
5. **External Factors:** The model does not account for external factors like weather conditions, supply chain delays, or political influences, which could affect the project's timeline and resource needs.

## 1.5 Methodology of Problem solving

The methodology for optimizing manpower allocation in rail construction projects centers around the application of **Linear Programming (LP)** techniques. The primary objective is to develop a mathematical model that minimizes the cost of labor while meeting project deadlines. The approach begins with a thorough understanding of the project's tasks, each of which requires specific labor resources over a defined period. Data collection is a crucial first step, involving gathering information on task priorities, available workforce, labor costs, and time constraints. This data is then used to formulate an LP model, with variables representing the workforce allocation for each task and constraints based on labor availability, project deadlines, and budget limits.

The LP model is designed with the following key components:

- **Objective Function:** The cost function, which aims to minimize the total labor cost across the entire project.
- **Constraints:** These include workforce availability, project timeline, task dependencies, and budget limitations.
- **Decision Variables:** Represent the number of workers assigned to each task, ensuring the optimal allocation based on the objective function.

To solve this model, optimization tools such as **Simplex Method** or specialized LP solvers (e.g., Gurobi, CPLEX) are employed. The solution obtained provides the optimal manpower distribution that minimizes costs and meets project constraints. A sensitivity analysis is conducted to examine how changes in input parameters, such as labor availability or task durations, affect the model's output. This step helps in understanding the robustness of the solution under different real-world scenarios.

By integrating LP with real-world rail construction data, the methodology ensures practical applicability, enabling construction managers to make data-driven decisions for efficient workforce allocation. Furthermore, the model can be iteratively refined as new data becomes available, offering flexibility to adapt to evolving project needs.

**CHAPTER 2**  
**LITERATURE SURVEY**

Employee or labor scheduling is a complex problem involving the allocation of an appropriate number of workers to tasks while balancing factors such as fluctuating labor availability, job requirements, and cost constraints. [2] presents a methodical approach to solving this challenge using linear programming techniques. The proposed model identifies and analyzes tasks, estimates workforce needs in terms of size and quality, and calculates expected labor expenses. It also considers the dynamic nature of labor availability, including part-time and full-time workers, and adapts schedules weekly to align with changing service demands. By systematically organizing tasks and optimizing labor costs while accommodating worker preferences, the study provides a logical framework for efficient workforce management in construction companies. This approach not only ensures the smooth running of projects but also minimizes labor costs and maximizes operational efficiency, making it a valuable tool for addressing labor scheduling problems in resource-constrained environments. Chen et al. demonstrates the ability of mixed integer programming models coupled with problem decomposition techniques to develop an algorithm that is capable of for manpower supply planning under mixed deterministic and stochastic demands [5]. Espiritu et al. (2020) expand on the use of LP by focusing on energy consumption optimization within rail transportation systems. Their research highlights how LP can be employed not only for labor management but also for enhancing overall operational efficiency through energy savings. By optimizing energy use, rail systems can reduce operational costs while improving sustainability, thereby supporting the dual goals of efficiency and environmental responsibility [4].

The integration of advanced methodologies in project management is explored by Kanakaris et al. (2020), who combine machine learning with OR techniques. Their study underscores the potential for predictive analytics to enhance decision-making in resource allocation, indicating a future direction for research in manpower optimization. By leveraging data-driven approaches, organizations can refine their labor management strategies and respond more effectively to changing operational demands [6]. Agarana et al. (2020) focus on the application of OR techniques in optimizing urban rail transportation, specifically in emerging countries. Their research emphasizes the unique challenges faced by these systems, including resource limitations and fluctuating demand patterns. By utilizing OR methodologies, they demonstrate how organizations can improve resource allocation and scheduling to enhance service delivery [1]. Liu et al. (2024) introduce a cost and passenger-responsible optimization method for planning additional high-speed train operations during peak periods. This study

emphasizes the importance of aligning manpower resources with passenger demand to improve service quality and operational efficiency. By incorporating passenger dynamics into operational planning, the research highlights the necessity for adaptable workforce strategies in response to peak demand situations [7].

Qi et al. (2023) present an integer linear programming model for integrated train stop planning and timetabling, addressing time-dependent passenger demand. Their findings reinforce the need for flexible scheduling practices that accommodate varying demand levels, which directly impacts manpower requirements. This model provides a comprehensive approach to ensuring that train services align with passenger needs, thereby optimizing labor allocation [8]. Zhao et al. (2023) delve into fluctuating demand-oriented optimization of train line planning, further illustrating the impact of demand variability on operational practices. Their study highlights the necessity for adaptable labor management strategies to respond effectively to changing passenger volumes, reinforcing the importance of integrating demand forecasting into workforce planning [10]. The optimization of railway systems has been a critical focus of research due to the increasing demands for efficiency and sustainability. In high-speed railways, strategies such as integrating train-stop planning and timetabling have demonstrated the potential to enhance operational efficiency by responding dynamically to passenger demands through mixed-integer linear programming model [12]. This approach not only minimizes travel time and unfulfilled passenger demand but also highlights the significance of data-driven decision-making in complex transport systems. Building on this, the present study extends the principles of optimization to workforce management in the railway construction and maintenance sectors. By employing Linear Programming techniques, it explores how aligning manpower allocation with project requirements can minimize operational costs while addressing resource constraints. This interconnected focus on optimizing both service delivery and resource management underscores the importance of Operations Research in advancing urban and high-speed railway systems.

M.C. Agarana et al. highlights the benefits of optimizing urban rail transportation in emerging countries using operational research techniques. The results highlight the effectiveness of Linear Programming (LP) to optimize the systems leading to lower costs, carbon emissions as well as economic development and eradication of poverty in emerging countries. [14]. The transportation model applied to Covenant University [9] leverages Linear Programming to address inefficiencies

in students' daily commutes between hostels and lecture rooms, a critical aspect of campus logistics. The study employs a structured approach using methods such as the North-West Corner, Least Cost, and Vogel's approximation techniques to determine an initial feasible solution, followed by the MODI method to optimize it further. This step-by-step application of linear programming ensures that the total time spent traveling is minimized, thereby enhancing the overall efficiency of campus transportation.

A novel integrated approach by Qi et al. (2023) toward the training of stop planning and timetabling with time-dependent passenger demand through an Integer Linear Programming model resulted in neglecting time-dependency, which led to more than 30% of passengers leaving outside preferred time intervals. [15] The study [13] supports the ideas presented in the paper by emphasizing the importance of optimizing resource allocation under constraints, particularly in dynamic rail transportation environments. It focuses on manpower optimization in railway projects and extends these principles by demonstrating how resource optimization, including labor and operational assets, can adapt to fluctuating demands.

Finally, Chhabra (2020) provides a historical perspective on operational research, detailing its significance during World War II. While not directly focused on rail transportation, this paper contextualizes the evolution of OR methodologies and their foundational role in contemporary applications, including manpower optimization in various sectors [3].



**CHAPTER 3**  
**SOFTWARE REQUIREMENTS**  
**SPECIFICATION**

## 3.1 Software Requirements Specifications

The Software Requirements Specification (SRS) for the Railway Network Optimization System outlines the essential functionalities, interfaces, performance expectations, and constraints of the system. This specification serves as a comprehensive blueprint for the design, development, and deployment phases, ensuring all stakeholders have a clear understanding of the system's scope and capabilities.

The system is designed to provide a user-friendly platform where users can upload railway network data, define optimization objectives (such as shortest path, minimum time, or least cost), and receive optimized routes through advanced algorithms. The results will be visually presented in graph and map formats, and users will have the ability to download outputs in various formats.

Key sections of the SRS include functional requirements (detailing core user-facing features), external interface requirements (highlighting how users will interact with the system), non-functional requirements (like performance, usability, and security), system requirements (including database and hardware specifications), and the chosen SDLC model supported by relevant diagrams (use case, data flow, component).

### 3.1.1 Assumptions & Dependencies

This section identifies external factors and environmental conditions assumed to hold true for the system to function correctly, as well as dependencies on other systems, libraries, or frameworks.

- **Assumption: Valid and Consistent Input Data**

The system assumes that users will upload railway network data in the correct format (CSV or Excel), with clearly defined station names, unique identifiers, valid route connections, and accurate attributes like distance, time, or cost. It is expected that data is clean and free from missing or corrupted values.

- **Assumption: Single Optimization Objective per Run**

For each optimization process, the user is assumed to select only one objective (e.g., shortest path, least cost, or minimal time). Multi-objective optimization is not supported in the current scope.

- **Dependency: Streamlit Framework**

The user interface of the system is built on the Streamlit framework. Any major changes, deprecations, or failures in Streamlit could impact the stability or availability of the interface.

- **Dependency: Python Libraries (NetworkX, Pandas, Plotly, etc.)**

The system relies on a set of Python libraries for core functionalities:

- **NetworkX** for graph-based algorithms (like shortest path computation),
- **Pandas** for reading and handling tabular data,
- **Plotly/Matplotlib** for visualization,
- **Openpyxl/XlsxWriter** for exporting to Excel. Any compatibility issues, deprecated methods, or version mismatches could disrupt the functioning of these features.

- **Dependency: Hosting Environment Compatibility**

The deployment environment (cloud-based or local) must support the execution of Python scripts and allow file uploads and downloads. The system is assumed to run on machines with internet access and compatible operating systems.

- **Assumption: Moderate Dataset Size**

It is assumed that the uploaded datasets will remain within a reasonable size to ensure quick computations and smooth rendering. Exception handling for very large datasets will be added in future versions but is currently limited.

## 3.2 Functional Requirements

### 3.2.1 Upload Railway Network Data (Stations, Routes)

The system must allow users to upload railway network data files in CSV or Excel formats. These files should include structured information about station names, route IDs, and key parameters such as distances, travel time, and cost. A robust parser should validate the input format and provide feedback for errors such as missing columns, unsupported characters, or inconsistent formatting.

Upon successful upload, the system should internally convert the data into a graph structure, where nodes represent stations and edges represent the routes connecting them. This structure becomes the foundational data model upon which optimization algorithms will operate. Data should be retained temporarily in session memory and automatically cleared upon system reset or when a new file is uploaded.

### 3.2.2 Define Optimization Objective (Shortest Path, Time, Cost)

The user must be able to specify the optimization goal from a set of predefined objectives: shortest distance, minimum time, or lowest cost. This input will directly influence which algorithm or cost metric is applied during the route optimization process. The objective will be selected via a user interface component such as a radio button or dropdown menu.

Once selected, this objective remains constant throughout the optimization process, unless the system is reset or a new objective is defined. The backend logic should dynamically adjust the weightage or cost function used in the algorithm based on the selected criterion, ensuring

optimal route calculation as per the user's intent.

### **3.2.3 Run Optimization Algorithm**

After uploading the network and selecting the objective, users can trigger the optimization process. The system should use efficient graph traversal algorithms like Dijkstra's, A\*, or Bellman-Ford depending on the selected metric and network complexity. Performance and accuracy are critical—results should be delivered quickly, even for large datasets.

During the computation, the interface should show real-time progress indicators or loading spinners. Once the algorithm completes, results are stored in memory for visualization and download. The algorithm module should also handle invalid cases, such as disconnected graphs or unreachable nodes, with appropriate user notifications.

### **3.2.4 Graphical Visualization of Network**

Once the optimization process is complete, users should be able to explore the results through an interactive graphical visualization of the railway network. The visualization will display stations as nodes and routes as edges, making it easy to identify how different stations are connected and how the optimized routes relate to the existing network. The system should leverage visualization libraries such as Plotly or NetworkX to generate an interactive graph, where users can zoom in, hover over nodes to view station details, and highlight specific paths to better understand the network. This feature is crucial for users who need to visually inspect the optimization results and verify that the routes align with their expectations. It also helps users identify potential issues with the optimization process, such as unexpected detours or inefficient routes. In addition, the graph should allow for easy navigation and exploration, ensuring that users can focus on specific areas of the network or test different optimization scenarios interactively.

### **3.2.5 Display Optimized Results (Text, Graph, Map)**

In addition to the graphical visualization, the system will provide a textual summary of the optimized results, including key details such as the total distance, time, or cost of the optimized route(s). This summary will allow users to quickly assess the outcomes without needing to interpret the graph. The text-based summary can also include additional information such as intermediate station stops, the time required for each segment of the route, and the cost per segment. Additionally, a map-based visualization will help users see the optimized routes overlaid on a geographic map, providing spatial context to the results. The map can display the paths in different colors or styles, helping users easily distinguish between multiple possible routes or compare the optimized route with alternative options. This multi-format approach to displaying results ensures that users can access the information in the format that best suits their needs, whether they prefer a quick summary or a detailed visual representation.

### **3.2.6 Download Optimized Output (CSV/Excel)**

To enable offline analysis or further documentation, users should have the option to download the results of their optimization process in structured formats like CSV or Excel. The downloadable file should contain all relevant data, including station names, route IDs, travel times, distances, costs, and other optimization parameters used during the process. This functionality is especially important for users who wish to share results with colleagues, perform further analysis using spreadsheet software, or keep a record of their optimization scenarios for future reference. The system should ensure that the exported file is properly formatted and includes clear headers and organized columns to make it easy for users to work with. Additionally, users should be able to download the results of any scenario they've run, including both the optimal and non-optimal solutions, allowing for comprehensive analysis and comparison.

### **3.2.7 Reset & Rerun Optimization**

To allow users to experiment with different data inputs or optimization goals, the system must include a feature to reset the current session and rerun the optimization process. This feature is important for scenarios where users want to modify their dataset or change the optimization objective without starting from scratch. The reset functionality should clear all uploaded data and parameters, enabling users to upload new datasets or adjust their existing input and reinitiate the optimization process. This feature enhances the platform's flexibility, allowing for iterative testing and refining of optimization strategies. Whether users are exploring different routes, testing various optimization objectives, or comparing alternative configurations, the ability to reset and rerun the process ensures they can refine their plans based on the results of previous iterations.

## **3.3 External Interface Requirements**

### **3.3.1 Streamlit Web Interface**

The core user interface of the system will be built using the Streamlit framework, which offers a lightweight, Python-based environment for deploying interactive web applications. Streamlit allows rapid development of UI components while tightly integrating with Python logic, making it ideal for data-driven tools like the railway optimizer. The framework supports automatic refreshing of outputs, seamless integration with data visualizations, and session state management for improved user experience.

Users will interact with the application through a browser-based interface hosted on a local server or cloud platform. Streamlit will provide the structural layout, including the sidebar for

navigation, input sections for file uploads, and main panels for visual outputs. The interface should support dynamic rerendering based on user inputs, providing an intuitive and responsive interaction flow across all steps of the optimization process.

### 3.3.2 File Upload Interface (CSV/Excel Input)

A key part of the interface is the file upload module that allows users to submit railway network data in CSV or Excel format. This component will use Streamlit's built-in `file_uploader()` functionality, which supports drag-and-drop or click-to-select uploads. To ensure robust parsing, the module will automatically validate the file type, check for required columns, and preview the data structure.

Post-upload, the system will retain the data in session state to facilitate subsequent processing. If errors are found—such as missing headers or incorrect file types—users will receive immediate visual feedback. This upload module will serve as the entry point into the optimization pipeline, and its usability and error-handling will directly impact the system's reliability.

### 3.3.3 Interactive Widgets (Sliders, Dropdowns)

To define the optimization objectives and fine-tune parameters, the system will use Streamlit's interactive widgets such as sliders, dropdowns, and radio buttons. These controls allow users to select between optimization goals like shortest distance, minimum cost, or least travel time, and set constraints or weights if needed. They enhance flexibility and make the tool adaptable for varied use cases.

These UI elements are crucial for usability—they provide a no-code mechanism for configuring complex optimization settings. Changes made via widgets will instantly update backend computations and visualizations, thanks to Streamlit's reactivity. Widget values will also be stored in session state to preserve user selections between interactions or re-runs.

### 3.3.4 Graph & Map Visualization Interface

Visual feedback is critical in an optimization system, and this will be handled via Streamlit-compatible libraries such as Plotly, NetworkX (for graphs), and Leaflet (for map-based visualizations). These components will render the uploaded railway network and highlight optimized paths based on the selected criteria. The map interface, in particular, will provide spatial context if geolocation data is included in the dataset.

Users should be able to interact with the graph by zooming, hovering, and clicking on nodes

and edges for more details. The visualizations will refresh automatically with each optimization run or data upload, making them dynamic and responsive. These interfaces aim to bridge the gap between raw data and user understanding, enabling better decision-making.

### **3.4 Nonfunctional Requirements**

#### **3.4.1 Performance Requirements**

The system must be able to process and optimize large-scale railway networks with minimal latency. This includes efficient handling of datasets with hundreds of stations and thousands of routes. Optimization computations should complete within a reasonable time frame (ideally under 5 seconds for medium-sized networks), and the system should maintain responsiveness during data input, algorithm execution, and visualization rendering.

To ensure performance, the backend optimization logic will be built using optimized data structures and algorithms (e.g., Dijkstra's or A\* for shortest path). The use of asynchronous processing for file parsing and result display can further enhance speed. Streamlit's caching mechanisms will also be leveraged to avoid redundant computations during repeated runs with the same inputs.

#### **3.4.2 Safety Requirements & Security Requirements**

The system will operate in a controlled environment where data privacy and user safety are respected. Although the application may not involve sensitive personal data, standard data security practices such as input validation, file sanitization, and restricted file types (e.g., disallowing executable uploads) will be implemented. This ensures that malicious scripts or corrupted files cannot compromise system integrity.

In multi-user deployments (e.g., hosted versions), authentication mechanisms may be introduced to limit access. Additionally, rate limiting and session timeouts can be considered to prevent misuse or overload. All user-uploaded data will be stored temporarily in session memory and cleared after each session to reduce risk and enhance safety compliance.

#### **3.4.3 Usability and Accessibility**

The system interface will follow user-centered design principles to ensure it is intuitive and easy to navigate, even for users without technical backgrounds. Layouts will be clean and minimal, with tooltips, visual feedback, and status indicators guiding the user through the optimization process. Clear error messages and progress indicators will assist users during file uploads, configuration, and result retrieval.

Accessibility considerations will include color-contrast compliance, font readability, and

screen-reader compatibility. Streamlit allows for responsive design, which means users on smaller screens or assistive devices will still be able to interact effectively with the tool. These features collectively aim to make the tool usable for a diverse group of stakeholders, including railway planners, researchers, and students.

### 3.4.4 Availability and Deployment

The system should be available for use with minimal downtime and easy deployment procedures. For local use, it can be run as a standalone Python application with simple Streamlit commands. For cloud or organizational deployment, Docker containers or platforms like Heroku or Streamlit Cloud can be used for scalable, on-demand hosting.

Deployment scripts and requirements will be clearly documented, and version control will be maintained via Git. Future versions can introduce continuous integration/continuous deployment (CI/CD) pipelines to ensure smooth updates. System logs and basic health checks will be added to monitor uptime and detect deployment issues early, thereby improving overall reliability and availability.

## 3.5 System Requirements

### 3.5.1 Database Requirements

While the system is primarily designed to function on temporary, session-based data processing, it can optionally include a lightweight database system to store historical runs, user settings, or predefined network configurations. SQLite or PostgreSQL can be used to manage this data efficiently, depending on whether the deployment is local or online. These databases will allow persistent storage of station-route networks, user-uploaded files, optimization results, and timestamps of operations.

Such a database system enhances flexibility—users could revisit previous optimizations, analyze trends over time, or benchmark results under different objectives. Database design will include normalized tables for stations, routes, optimization objectives, and user metadata if needed. Indexing and efficient querying will be prioritized to keep retrieval fast even with large datasets.

### 3.5.2 Software Requirements

The core of the application will be built using Python due to its extensive ecosystem for data processing, optimization, and visualization. Key libraries include Streamlit (for the web



interface), Pandas and NumPy (for data manipulation), NetworkX (for graph-based operations), and optionally Plotly or PyDeck (for advanced visualizations). These tools ensure seamless development, interaction, and performance.

Additional requirements include a virtual environment for dependency management (e.g., via venv or conda), and a requirements file (requirements.txt) to ensure all dependencies are easily installed. Compatibility will be ensured with Python 3.8+ and tested across major platforms (Windows, Linux, macOS). All dependencies will be lightweight to allow smooth setup and reduce system bloat.

### 3.5.3 Hardware Requirements

Since the tool is not highly resource-intensive, it can be run on any standard computer with at least 4 GB RAM and a dual-core processor. These specifications are sufficient for processing medium-sized networks and generating visualizations in real-time. For larger networks or batch optimizations, systems with 8 GB+ RAM and SSD storage are recommended for better performance.

The system is compatible with basic hardware configurations, making it accessible to users in resource-constrained environments. No GPU is required since all computations are CPU-bound. For hosting in a cloud environment, minimal virtual machine specifications (e.g., 1 vCPU, 1 GB RAM) will suffice due to the light nature of the application. Scalability can be achieved through containerized deployments if needed.

## Optimizing Manpower Allocation in Rail Construction

---

# **CHAPTER 4**

## **SYSTEM DESIGN**

## 4.1 System Architecture

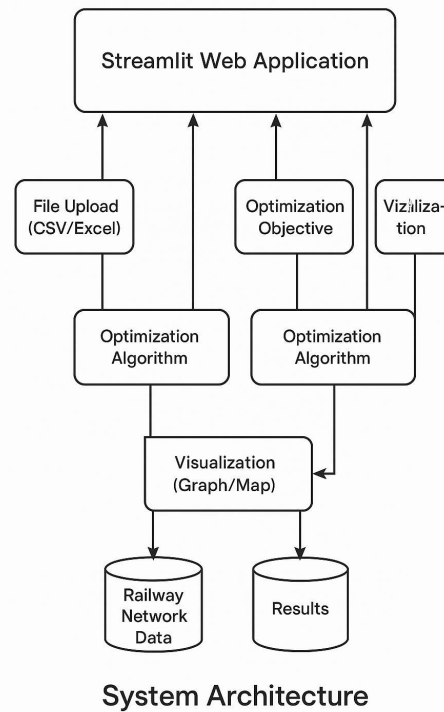


Figure 4.1: System Architecture

- User Interface Layer

Built using Streamlit for easy access via a web browser.

Includes file upload components, dropdowns, sliders, and result visualization tools (graphs, maps, tables).

- Application Logic Layer

Handles user input processing and validation.

Contains optimization algorithms (e.g., Dijkstra's, A\*) based on user-selected objectives like time, cost, or shortest path.

- Data Processing Layer

Parses CSV/Excel files containing railway network data.

Converts raw input into graph data structures (nodes as stations, edges as routes).

- Visualization Layer

Renders interactive graphs and maps of the railway network.

Displays optimized paths visually and in tabular formats.

- Storage Layer

Uses a lightweight database or in-memory storage to store session data, uploaded files, and computation outputs.

- Export/Download Module

Converts final optimized results into downloadable CSV or Excel format for users.

## 4.2 UML Diagrams

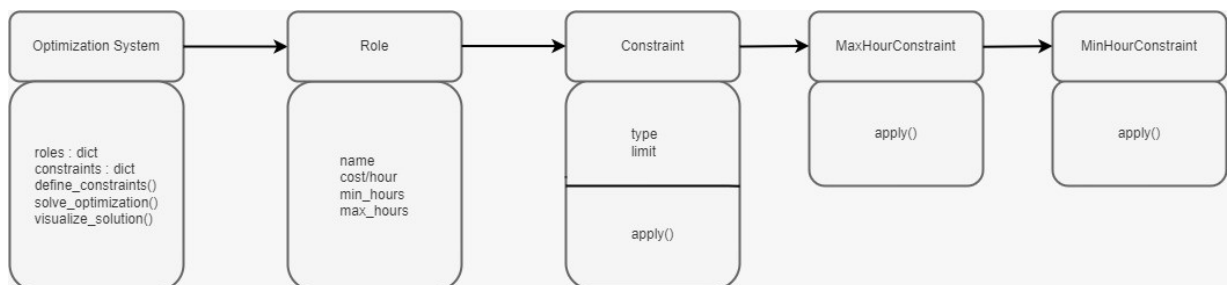
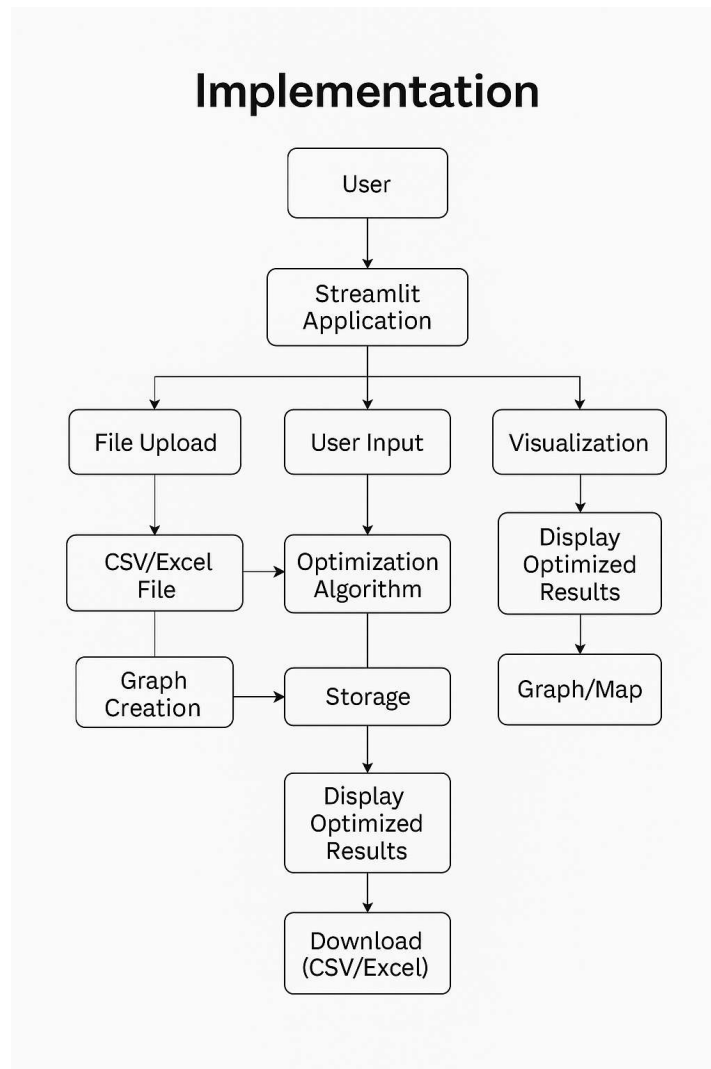
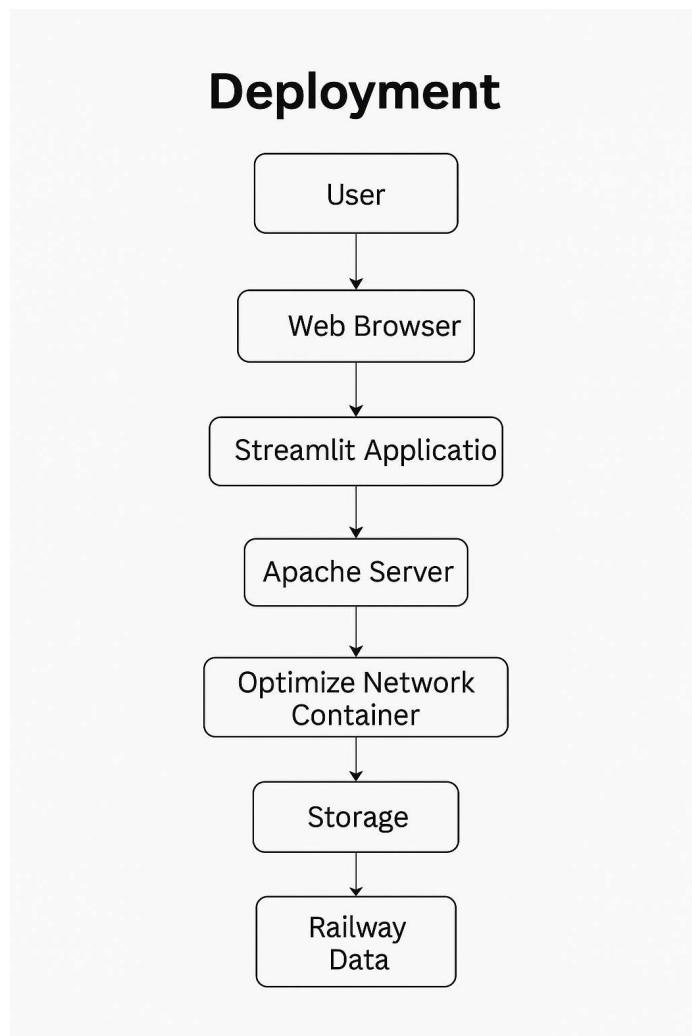


Figure 4.2: Use case diagram

## 4.3 Implementation Diagram



## 4.4 Deployment Diagram



# **CHAPTER 5**

## **PROJECT PLAN**



## 5.1 Project Estimate

### 5.1.1 Reconciled Estimates

- **Development Phase:**

Estimated **6 months** for developing the core functionality of the Linear Programming (LP) model, including data collection, formulation of the LP model, and running simulations.

This time allocation includes refining the model to account for real-world complexities such as labor costs, workforce availability, and project deadlines.

- **Testing and Evaluation:**

Estimated **2 months** for rigorous testing of the model, including sensitivity analysis, performance tests, and validation with real-world rail construction data.

- **Dashboard Development:**

Estimated **1 month** for developing and integrating the Looker Studio dashboard. This involves connecting the LP model results to the dashboard and creating interactive visualizations to track labor allocation, costs, and other metrics.

### 5.1.2 Project Resources

- **Technology Stack:**

- **Programming Languages:** Python (for the LP model), Numpy, Matplotlib, and optimization libraries such as SciPy for solving the LP model.
- **Optimization Software:** Gurobi or CPLEX for solving the LP problems.
- **Looker Studio:** For developing the dashboard to display and monitor key metrics related to labor allocation and project costs.
- **Collaboration Tools:** GitHub for version control, Google Drive for documentation, and Google Meet for communication.

- **Human Resources:**

- **Operations Research Specialist:** To create and optimize the LP model.
- **Data Analyst:** Responsible for gathering data (labor costs, workforce availability) and processing it for the LP model.
- **Software Developer:** For implementing the model and integrating it with existing infrastructure.

## **5.2 Risk Management**

Risk management is an essential aspect of our project to ensure its successful implementation. By proactively identifying and addressing potential risks, we can minimize their impact on the development and deployment of our application.

By proactively identifying these risks and implementing appropriate risk mitigation strategies, we aim to minimize the potential impact on the development, functionality, and security of our application. This proactive approach will contribute to smoother project execution and enhance the project's overall reliability and performance.

### **5.2.1 Risk Identification**

#### **1. Technical Risks:**

- Difficulty integrating the LP model with existing project management systems.
- Model inaccuracies due to changes in labor availability or costs.

#### **2. Business Risks:**

- Budget constraints leading to incomplete project functionalities.
- Project delays caused by unforeseen circumstances, such as resource unavailability.

#### **3. Resource Risks:**

- Lack of skilled personnel for model implementation and testing.
- Over-reliance on specific team members, leading to bottlenecks if they are unavailable.

### **5.2.2 Risk Analysis**

After identifying the risks, we have conducted a risk analysis to determine their potential impact and develop mitigation strategies:

1. Technical Risks:

- Mitigation: Regular testing and integration phases to ensure compatibility with real-world scenarios. Backup systems in place for data inconsistencies.

2. Business Risks:

- Mitigation: Regular communication with stakeholders to align on project scope, budget, and timelines.

3. Resource Risks:

- Mitigation: Cross-training team members to ensure that key tasks can be handled by multiple team members. Ensuring workload is distributed evenly.

## 5.3 Project Schedule

### 5.3.1 Project Task Set

- **Data Collection:** Gathering data on labor costs, workforce availability, and task durations from rail construction projects.
- **LP Model Development:** Designing the LP model, defining objective functions and constraints.
- **Dashboard Development:** Building the Looker Studio dashboard to visualize key metrics such as workforce allocation, costs, and task progress.
- **System Testing:** Testing the LP model with real-world scenarios and validating the results for accuracy and efficiency.
- **Documentation:** Writing detailed documentation covering model design, assumptions, and outcomes.
- **Project Review:** Continuous feedback from internal and external reviewers to ensure project goals are met.

5.3.2 Timeline Chart

This section represents summarized project timeline chart. It briefly explains the schedule of our entire project from inception to projected completion.

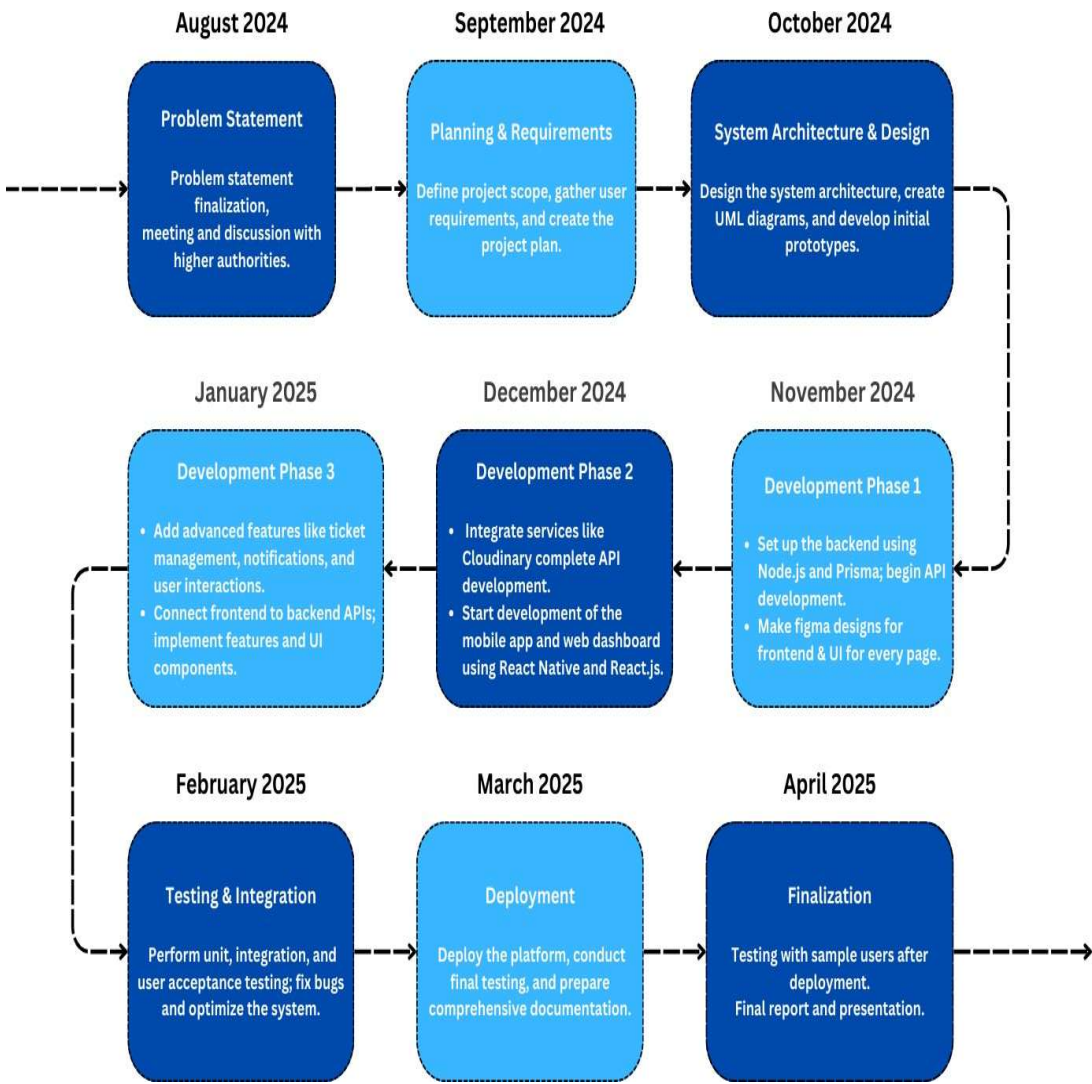


Figure 5.1: Summarized Timeline Chart

## 5.4 Team Organization

### 5.4.1 Team structure

- **Saniya Atalatti:** Project Leader UI/UX Designer.
- **Arnav Desai:** Operation Research Specialist
- **Shatakshi Chaudhari:** Data Analyst.
- **Hatim Talwara:** Dashboard Developer.
- **Dr. Girish Potdar:** Internal mentor and guide
- **Sameer Hase :** External guide

### 5.4.2 Management reporting and communication

- **Communication Tools:** Regular team meetings conducted through Google Meet or Zoom for updates. Daily progress is reported using Google Groups, and weekly check-ins with the internal mentor are scheduled.
- **Reporting:** Bi-weekly, we reported our progress to the internal mentor, and provided updates on work completed, challenges faced and ideas we came up with. The regular communication allowed us to stay aligned with project goals and deadlines of submissions.
- **Communication:** For regular communication in team members we used Google Groups and Whatsapp for quick updates, making it easy to share new information and get immediate feedback. Our communication with the mentor was structured around bi-weekly sessions and regular updates and tasks were given accordingly.

Thus, this structured approach to management reporting and communication helped us to maintain clarity, accountability and efficiency throughout the project.

## **CHAPTER 6**

### **OTHER SPECIFICATIONS**

## 6.1 Advantages

The following are the advantages of using our project.

- **Cost Minimization:** The LP model optimizes labor allocation, significantly reducing operational costs by efficiently distributing labor resources.
- **Scalability:** The model can be scaled to accommodate other construction projects, not just rail construction, making it adaptable to various infrastructure developments.
- **Data-Driven Decision Making:** The LP model provides stakeholders with data-driven insights into workforce allocation, helping to minimize delays and maximize resource efficiency.
- **Interactive Dashboard:** The Looker Studio dashboard provides an intuitive interface for stakeholders to visualize labor allocation, costs, and project timelines. This enhances data-driven decision-making by offering real-time insights into project performance.



## 6.2 Limitations

The accuracy of the model's output is highly dependent on the quality of input data, meaning any discrepancies in labor costs or availability can lead to skewed results. Additionally, while the Linear Programming (LP) model is useful, it may not fully account for real-world uncertainties such as sudden labor shortages or equipment breakdowns, limiting its flexibility. Larger projects with multiple tasks can also increase computational complexity, making real-time solutions more challenging to obtain. Furthermore, while the Looker Studio dashboard is interactive, its customizability may be restricted depending on the available data and specific metrics it tracks.

## 6.3 Applications

- **Rail Construction Projects:** Optimizing labor allocation in railway construction, track maintenance, and station renovations.
- **Other Infrastructure Projects:** The LP model can be applied to road construction, bridge maintenance, and other large-scale infrastructure initiatives where manpower optimization is required.
- **Visualization of Key Metrics:** The Looker Studio dashboard is a key tool for visualizing important project metrics such as workforce distribution, labor costs, and task completion rates. It serves as an interactive platform for project managers and stakeholders to monitor project progress in real-time.

# **CHAPTER 7**

## **CONCLUSION & FUTURE WORK**

## 7.1 Conclusion

The implementation of a Linear Programming model for manpower allocation in rail construction projects presents a significant opportunity to reduce operational costs and enhance project efficiency. Through the structured optimization of labor resources, this project demonstrates how mathematical modeling can address the challenges of resource scarcity, cost overruns, and project delays. The integration of real-world data ensures practical applicability, making this model a valuable tool for decision-makers in the rail construction industry. Additionally, the development of the Looker Studio dashboard provides an interactive and intuitive way to visualize project metrics, allowing stakeholders to make informed decisions in real-time.

## 7.2 Future Work

- Integrating real-time physiological insights into task allocation can transform workforce management by making it adaptive to human endurance and cognitive fluctuations. Future systems can use PPG-based fatigue detection to dynamically adjust task assignments, ensuring that cognitively demanding tasks are scheduled when an individual is most alert, while lower-effort tasks are assigned during periods of fatigue. Additionally, adaptive scheduling mechanisms could identify when an individual reaches a physiological threshold indicating mental exhaustion and automatically schedule a break. Instead of relying on fixed break times, the system would use real-time data to prompt individuals to rest exactly when needed, preventing burnout while maintaining productivity. This shift from static to real-time, user-aware task allocation would help create a more sustainable and humane work environment.
- Beyond individual workload management, collaborative task allocation can benefit from physiological insights by ensuring an even distribution of cognitive load across teams. If one individual shows signs of fatigue, tasks could be temporarily reallocated to colleagues who are in a better cognitive state, ensuring overall workflow efficiency without overburdening any one person. This approach would be particularly impactful in high-stakes environments such as healthcare, emergency response, and industrial settings, where errors caused by cognitive fatigue can have serious consequences. By embedding HCI principles into task allocation systems, future developments can ensure that workforce management is not only efficient but also fair, responsive, and human-centric, allowing individuals to perform at their best while maintaining their well-being.

# **CHAPTER 8**

## **APPENDIX**

## 8.1 Appendix A

### 8.1.1 Technical Feasibility

- **Technology Stack:** The project uses modern and scalable technologies like Python for the Linear Programming (LP) model development and optimization, along with Looker Studio for dashboard visualization. These technologies ensure high scalability and flexibility in managing and visualizing the labor allocation process.
- **Backend Architecture:** The integration of tools such as Gurobi or SciPy for optimization, and Looker Studio for interactive dashboards, ensures a robust and scalable platform. The backend system efficiently handles large amounts of data related to labor costs, workforce availability, and other metrics.
- **Looker Studio Dashboard:** The Looker Studio dashboard ensures the real-time visualization of key performance indicators, including labor allocation, project timelines, and costs. The integration of this dashboard enhances the technical viability of the project, providing decision-makers with an intuitive interface.

### 8.1.2 Operational Feasibility

- **User Testing:** The LP model has undergone testing with project managers in rail construction. Feedback from these tests indicates that the system is operationally feasible and effectively optimizes labor allocation and resource management.
- **Interface:** The Looker Studio dashboard, which provides users with a clear and real-time visual representation of project progress, has been well-received for its user-friendly interface. This significantly reduces manual labor planning efforts.
- **Process Efficiency:** The project's operations research framework, supported by the LP model and dashboard, streamlines manpower distribution and provides project managers with data-driven insights, making the overall process more efficient.

### 8.1.3 Economic Feasibility

- **Cost-Benefit Analysis:** A cost-benefit analysis was conducted, assessing the costs associated with platform development, implementation, and maintenance against the potential long-term benefits of labor cost optimization and project timeline efficiency.
- **Development and Maintenance Costs:** The initial development costs for the LP model and Looker Studio dashboard were evaluated against the projected savings in labor costs, reduced project delays, and enhanced decision-making capabilities.
- **Financial Viability:** The results of the cost-benefit analysis indicate that the savings from optimized labor allocation, coupled with reduced operational costs, make the project economically viable. The projected return on investment (ROI) justifies the initial costs.

## **8.2 Appendix B**

### **8.2.1 Survey Paper Details**

**Paper Status:** Accepted

**Name of Journal:** Volume 76 *Journal of Engineering and Technology Management (Jet-M)*.

## **8.3 Appendix C**

### **8.3.1 Plagiarism Report**

The plagiarism report for the project report has been attached herewith.

**Similarity:** 1%

# **CHAPTER 9**

## **REFERENCES**



- [1] Liu, Y., Cao, C., Feng, Z., Zhou, Y. (2024). A Cost and Passenger Responsible Optimization Method for the Operation Plan of Additional High-Speed Trains in a Peak Period.
- [2] Qi, J., Cacchiani, V., Yang, L., Zhang, C., Di, Z. (2023). An Integer Linear Programming model for integrated train stop planning and timetabling with time-dependent passenger demand.
- [3] Espiritu, N. M. D., Billones, R. K. C., Bugtai, N. T. (2020). Energy Consumption Optimization in Rail Transportation System using Linear Programming.
- [4] Kanakaris, N., Karacapilidis, N., Kournetas, G., Lazanas, A. (2020). Combining Machine Learning and Operations Research Methods to Advance the Project Management Practice.
- [5] Agarana, M. C., Anake, T. A., Okagbue, H. I. (2020). Optimization of Urban Rail Transportation in Emerging Countries Using Operational Research Techniques.
- [6] Zhao, C., Chen, J., Zhang, X., Wang, Z., Wu, P., Cui, Z. (2023). Fluctuating Demand-Oriented Optimization of Train Line Planning
- [7] Chhabra, P. (2020). Operational Research: During World War II.

[8] M. Zhou, L. Zhang, and K.C. Lam, "Manpower Planning for MTR Carriage Assembly Using Integer Programming," \emph{Journal of Rail Transportation}, 2020.

[9] Y. Li, B. Han, R. Yang, and P. Zhao, "Integrated Optimization of Stop Planning and Timetabling for High-Speed Railways," \emph{Applied Sciences}, vol. 13, no. 1, 2023.

[10] X. Zhang, Z. Wang, P. Wu, and Z. Cui, "Fluctuating Demand-Oriented Optimization of Train Line Planning," \emph{Applied Sciences}, vol. 12, no. 18, 2022.

[11] N. M. D. Espiritu, R. K. C. Billones, and N. T. Bugtai, "Energy Consumption Optimization in Rail Transportation Using Linear Programming," \emph{International Journal of Rail Transport}, 2020.

[12] M. C. Agarana, T. A. Anake, and H. I. Okagbue, "Optimization of Urban Rail Transportation Using Operations Research Techniques," \emph{Journal of Transportation Management}, 2020.

[13] J. Qi, V. Cacchiani, L. Yang, C. Zhang, and Z. Di, "Integer Linear Programming for Integrated Train Stop Planning and Timetabling," \emph{Transportation Science}, vol. 57, 2023.

LIST OF ABBREVIATIONS

| Abbreviation | Illustration                |
|--------------|-----------------------------|
| LP           | Linear Programming          |
| PPG          | Photo Pletheshmogram        |
| MLOps        | Machine learning operations |
| AI           | Artificial Intelligence     |